

2.1 Definition and Properties of LTI Systems

Aim

- To understand the concept of Linear Time-Invariant (LTI) systems.
- To verify the properties of linearity and time invariance.
- To distinguish between LTI and non-LTI systems using graphical analysis.

Problem 1

Problem Statement

Consider the system

$$y(t) = 3x(t)$$

Verify whether the given system satisfies the property of linearity by plotting the input and output signals.

```
In [9]: import numpy as np
import matplotlib.pyplot as plt

# Time axis
t = np.linspace(-5,5,1000)

# Input signal
x = np.sin(t)

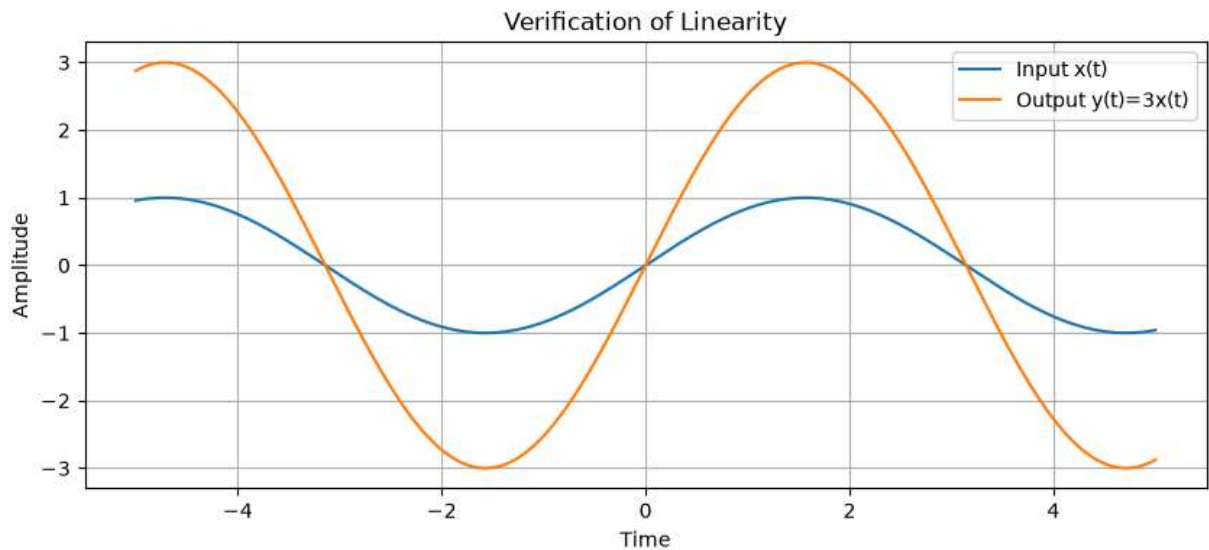
# Output signal
y = 3*x

plt.figure(figsize=(10,4))

plt.plot(t,x,label="Input x(t)")
plt.plot(t,y,label="Output y(t)=3x(t)")

plt.title("Verification of Linearity")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()

plt.show()
```



Observation

- The output signal is three times the input signal.
- The waveform shape remains unchanged.
- The system satisfies the linearity property.

Problem 2

Problem Statement

Consider the system

$$y(t) = x(t) + 5$$

Determine whether the system is linear by comparing the input and output signals.

```
In [10]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-5,5,1000)

x = np.sin(t)

y = x + 5

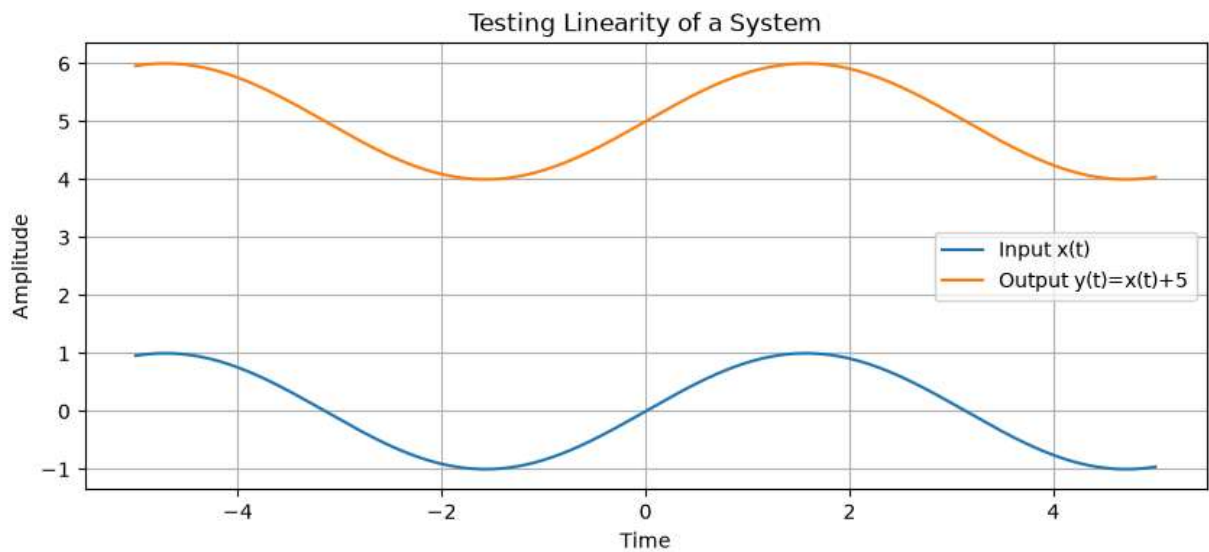
plt.figure(figsize=(10,4))

plt.plot(t,x,label="Input x(t)")
plt.plot(t,y,label="Output y(t)=x(t)+5")

plt.title("Testing Linearity of a System")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
```

```
plt.legend()
```

```
plt.show()
```



Observation

- The output signal contains a constant offset of 5.
- The output is not directly proportional to the input.
- Hence, the system does **not** satisfy the linearity property.

Problem 3

Problem Statement

Consider the system

$$y(t) = x(2t)$$

Investigate whether the system is time-invariant by comparing the original signal with the transformed signal.

```
In [11]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-5,5,1000)

x = np.cos(t)

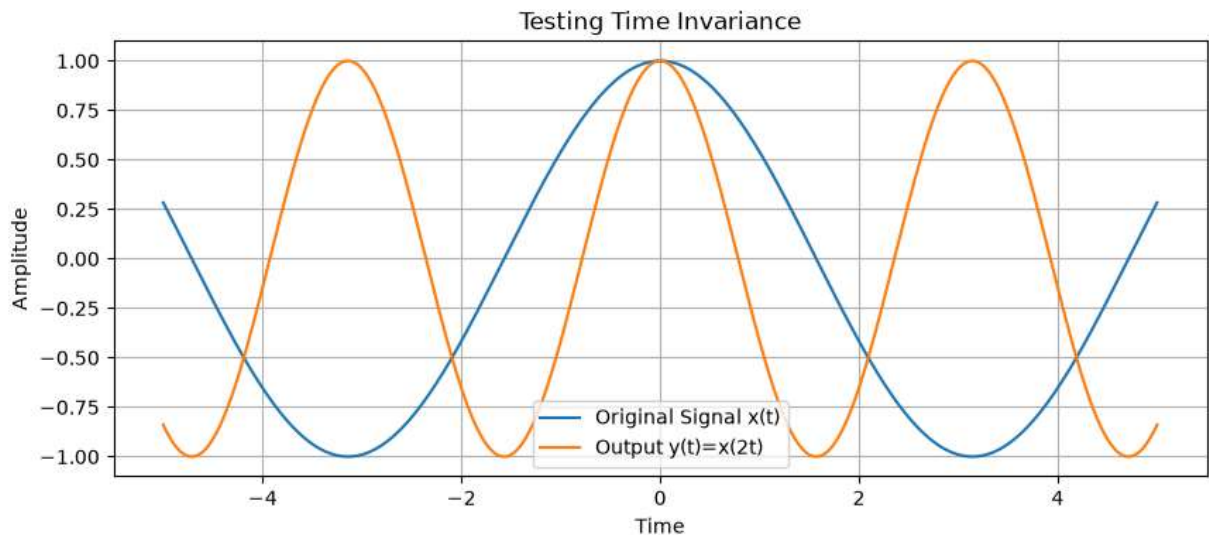
y = np.cos(2*t)

plt.figure(figsize=(10,4))

plt.plot(t,x,label="Original Signal x(t)")
plt.plot(t,y,label="Output y(t)=x(2t)")
```

```
plt.title("Testing Time Invariance")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()

plt.show()
```



Observation

- The output signal is compressed along the time axis.
- The system changes the time scale of the input.
- Therefore, the system is **not time-invariant**.

Conclusion

In this section, the basic properties of LTI systems were studied. A linear system scales the input without altering its shape, whereas adding a constant makes a system non-linear. It was also observed that time scaling changes the signal characteristics, indicating that the system is not time-invariant.

2.2 Convolution Integral (Continuous-Time) and Convolution Sum (Discrete-Time)

Aim

- To understand the concept of convolution.
- To perform convolution of continuous-time and discrete-time signals.

- To verify convolution results using Python.
- To compare manual and built-in convolution methods.

Problem 4

Problem Statement

Consider the discrete-time signals

$$x[n] = \{1, 2, 1\}$$

$$h[n] = \{1, 1, 1\}$$

Compute their convolution using the built-in NumPy function and plot the input signals along with the output sequence.

```
In [12]: import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,1])
h = np.array([1,1,1])

y = np.convolve(x,h)

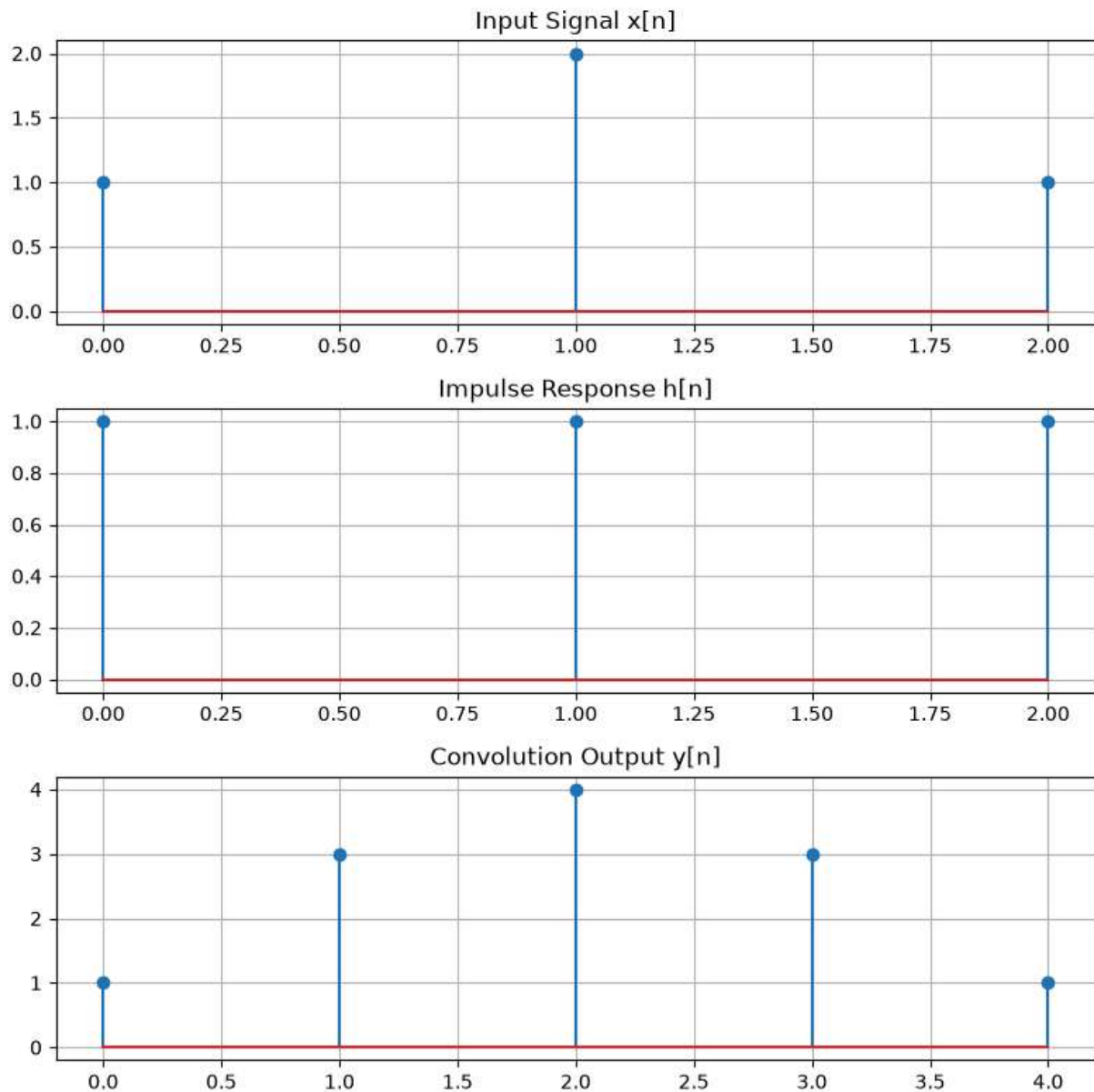
fig, ax = plt.subplots(3,1,figsize=(8,8))

ax[0].stem(range(len(x)),x)
ax[0].set_title("Input Signal x[n]")
ax[0].grid(True)

ax[1].stem(range(len(h)),h)
ax[1].set_title("Impulse Response h[n]")
ax[1].grid(True)

ax[2].stem(range(len(y)),y)
ax[2].set_title("Convolution Output y[n]")
ax[2].grid(True)

plt.tight_layout()
plt.show()
```



Observation

- The output sequence has a length of $\mathbf{N + M - 1}$.
- Convolution combines the effect of the input signal and the impulse response.
- NumPy computes the convolution efficiently using the built-in function.

Problem 5

Problem Statement

Consider the continuous-time signals

$$x(t) = e^{-t}u(t)$$

and

$$h(t) = e^{-2t}u(t)$$

Plot both signals and compute their convolution numerically.

```
In [13]: import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,5,500)

x = np.exp(-t)

h = np.exp(-2*t)

dt = t[1]-t[0]

y = np.convolve(x,h)*dt

tc = np.linspace(0,10,len(y))

plt.figure(figsize=(10,5))

plt.plot(tc,y,label="Convolution Output")

plt.xlabel("Time")

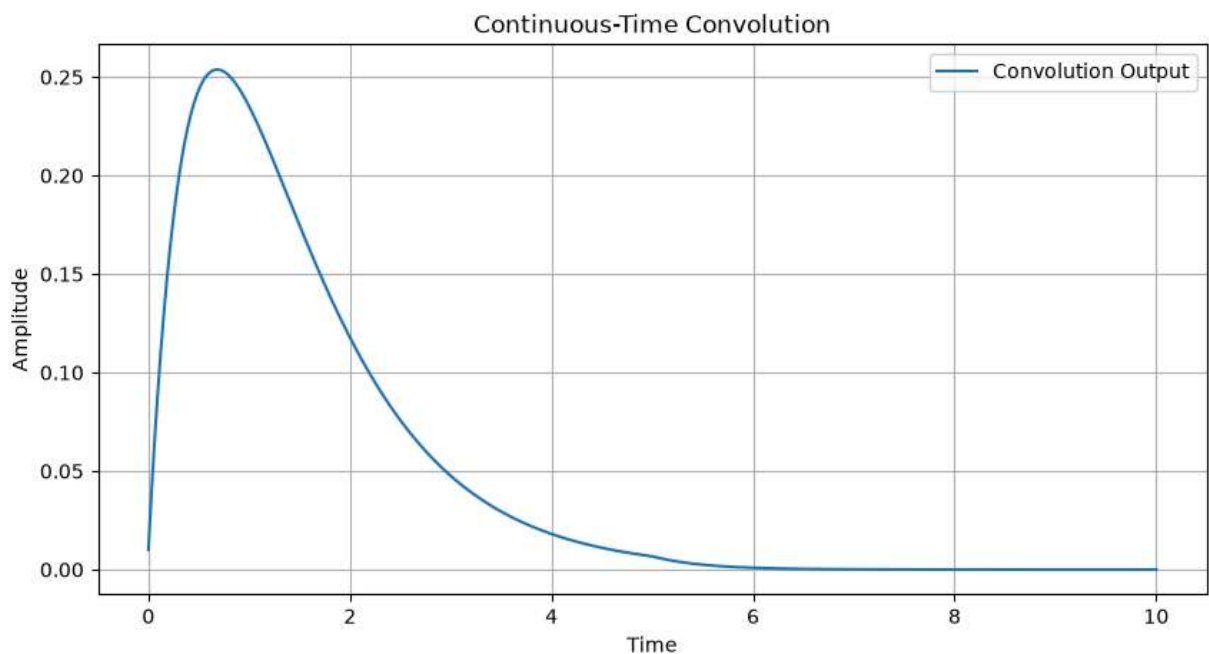
plt.ylabel("Amplitude")

plt.title("Continuous-Time Convolution")

plt.grid(True)

plt.legend()

plt.show()
```



Observation

- The convolution output is smoother than the original signals.
- The output represents the response of the system to the given input.
- Continuous-time convolution is approximated numerically using discrete samples.

Problem 6

Problem Statement

Perform the convolution of the following discrete-time sequences **without using** the built-in `numpy.convolve()` function.

$$x[n] = \{1, 2, 1\}$$

$$h[n] = \{1, 1, 1\}$$

```
In [14]: import numpy as np
import matplotlib.pyplot as plt

x = [1,2,1]
h = [1,1,1]

N = len(x)
M = len(h)

y = [0]*(N+M-1)

for i in range(N):
    for j in range(M):
        y[i+j] += x[i]*h[j]

plt.figure(figsize=(8,4))

plt.stem(range(len(y)),y)

plt.title("Manual Convolution using Nested Loops")

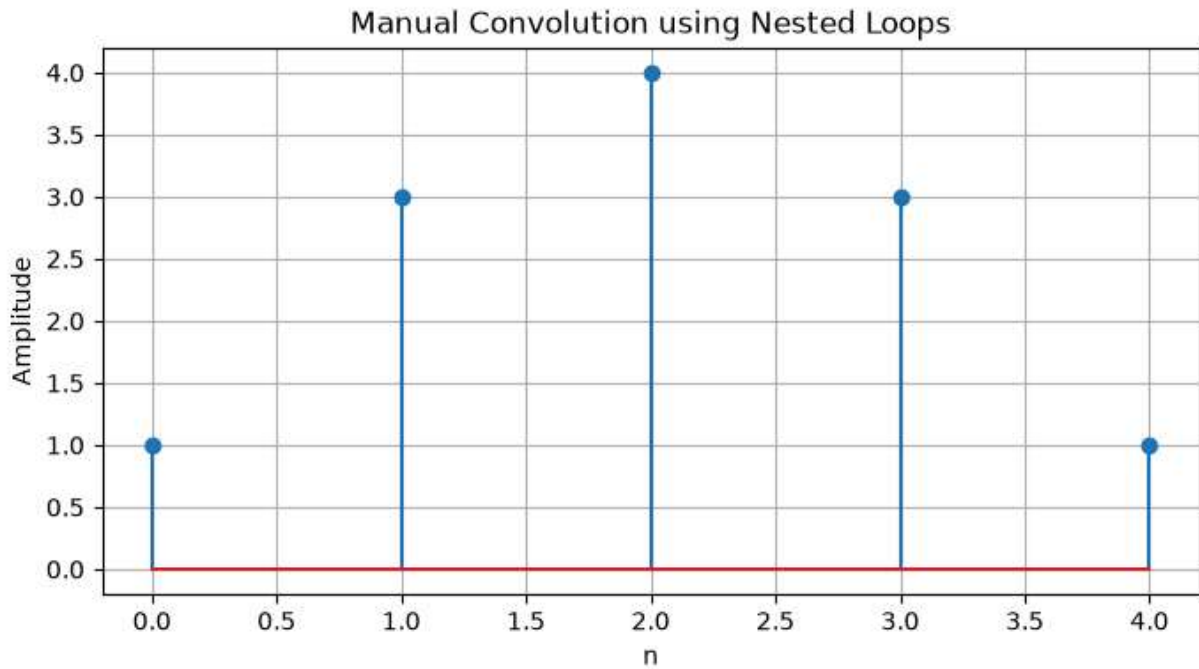
plt.xlabel("n")

plt.ylabel("Amplitude")

plt.grid(True)

plt.show()

print("Convolution Output =",y)
```



Convolution Output = [1, 3, 4, 3, 1]

Observation

- Manual convolution gives the same result as the built-in function.
- Nested loops implement the mathematical definition of convolution.
- This verifies the correctness of the convolution operation.

Problem 7

Problem Statement

Compare the results obtained using the manual convolution method and the NumPy built-in convolution function for the given sequences.

```
In [15]: import numpy as np
import matplotlib.pyplot as plt

x=[1,2,1]
h=[1,1,1]

manual=[0]*(len(x)+len(h)-1)

for i in range(len(x)):
    for j in range(len(h)):
        manual[i+j]+=x[i]*h[j]

builtin=np.convolve(x,h)

plt.figure(figsize=(9,4))
```

```

plt.stem(range(len(manual)),manual,linewidth='b-',markerfmt='bo',basefmt=' ')
plt.stem(range(len(builtin)),builtin,linewidth='r--',markerfmt='rs',basefmt=' ')

plt.legend(["Manual", "NumPy"])

plt.title("Comparison of Manual and Built-in Convolution")

plt.xlabel("n")

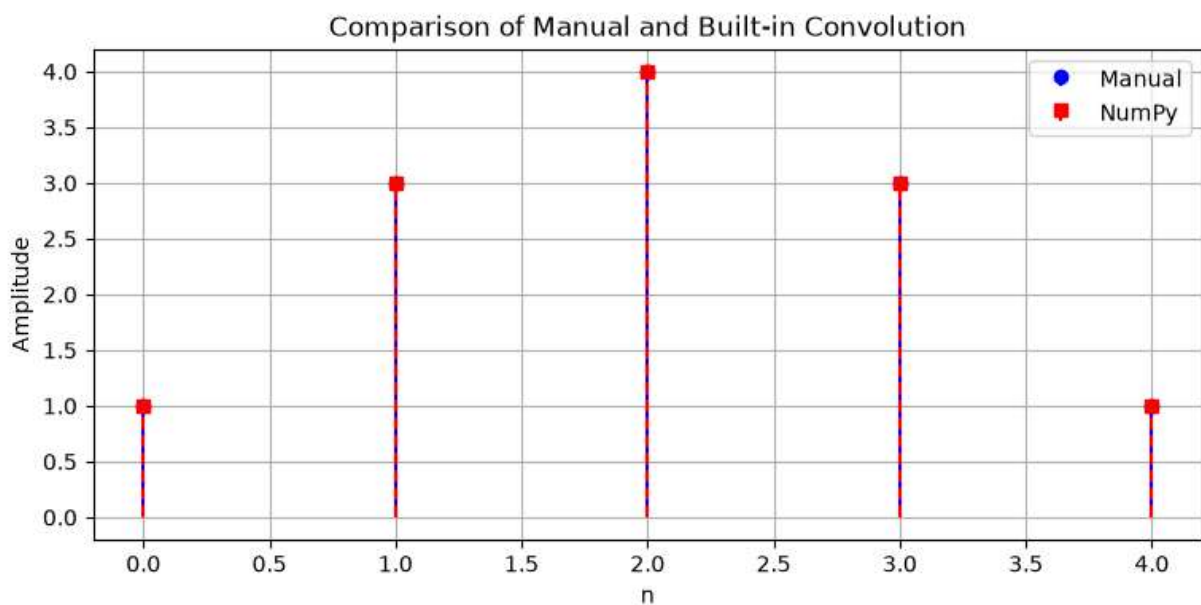
plt.ylabel("Amplitude")

plt.grid(True)

plt.show()

print("Manual :",manual)
print("NumPy  :",builtin)

```



Manual : [1, 3, 4, 3, 1]

NumPy : [1 3 4 3 1]

In []:

Observation

- Both methods produce identical convolution results.
- The built-in function is computationally efficient.
- Manual convolution provides a better understanding of the convolution process.

Conclusion

Convolution is a fundamental operation used to determine the output of an LTI system. In this section, both continuous-time and discrete-time convolution were performed. The

results obtained using manual computation matched those obtained from the built-in NumPy function, confirming the correctness of the convolution process.

Problem 8

Problem Statement

Consider the discrete-time signals

$$x[n] = \{1, 2, 1\}$$

$$h[n] = \{1, 1\}$$

Perform convolution graphically by plotting the input signals and the convolution output.

```
In [16]: import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,1])
h = np.array([1,1])

y = np.convolve(x,h)

plt.figure(figsize=(10,6))

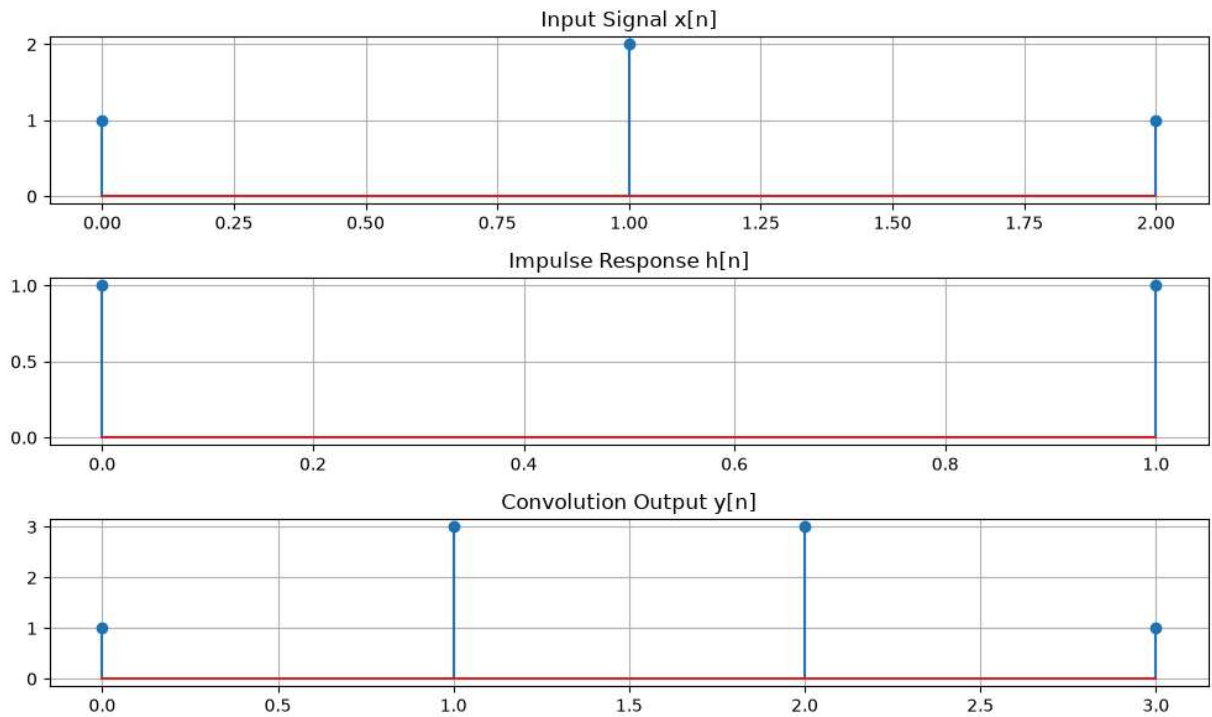
plt.subplot(3,1,1)
plt.stem(range(len(x)),x)
plt.title("Input Signal x[n]")
plt.grid(True)

plt.subplot(3,1,2)
plt.stem(range(len(h)),h)
plt.title("Impulse Response h[n]")
plt.grid(True)

plt.subplot(3,1,3)
plt.stem(range(len(y)),y)
plt.title("Convolution Output y[n]")
plt.grid(True)

plt.tight_layout()

plt.show()
```



Observation

- The convolution output is obtained by sliding one sequence over the other.
- The output length equals $(N+M-1)$.
- Graphical visualization helps understand the convolution process.

Problem 9

Problem Statement

Verify that the analytical convolution result matches the result obtained using Python.

```
In [17]: import numpy as np

x=[1,2,1]
h=[1,1]

analytical=[1,3,3,1]

python_result=np.convolve(x,h)

print("Analytical Result :",analytical)

print("Python Result      :",python_result)

if np.array_equal(analytical,python_result):
    print("\nResult Verified Successfully.")
```

```
else:  
    print("\nResults do not match.")
```

Analytical Result : [1, 3, 3, 1]

Python Result : [1 3 3 1]

Result Verified Successfully.

Observation

- The analytical computation matches the Python result.
- This verifies the correctness of the convolution operation.

2.3 Impulse Response and Step Response

Aim

- To understand impulse response and step response.
- To compare both responses graphically.
- To study their relationship.

Problem 10

Problem Statement

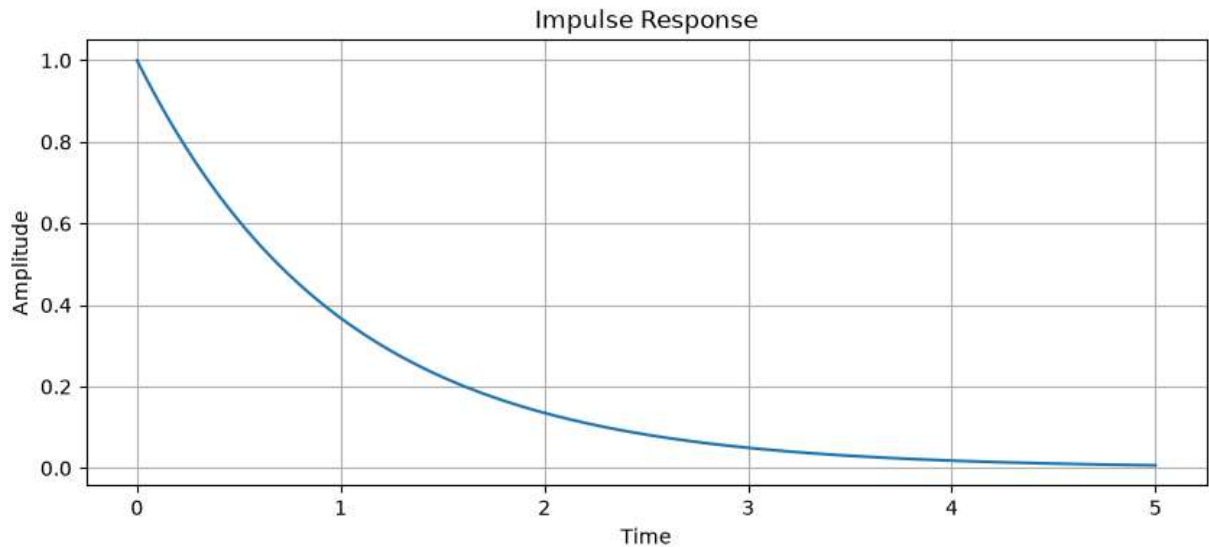
Plot the impulse response

$$h(t) = e^{-t}u(t)$$

where $u(t)$ is the unit step function.

```
In [18]: import numpy as np  
import matplotlib.pyplot as plt  
  
t=np.linspace(0,5,500)  
  
h=np.exp(-t)  
  
plt.figure(figsize=(10,4))  
  
plt.plot(t,h)  
  
plt.title("Impulse Response")  
  
plt.xlabel("Time")  
  
plt.ylabel("Amplitude")  
  
plt.grid(True)
```

```
plt.show()
```



Observation

- The impulse response decreases exponentially.
- It characterizes the complete behavior of an LTI system.

Problem 11

Problem Statement

Obtain the step response from the impulse response using cumulative summation and compare the two responses.

```
In [19]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(0,5,500)

dt=t[1]-t[0]

h=np.exp(-t)

step=np.cumsum(h)*dt

plt.figure(figsize=(10,4))

plt.plot(t,h,label="Impulse Response")

plt.plot(t,step,label="Step Response")

plt.title("Impulse Response vs Step Response")

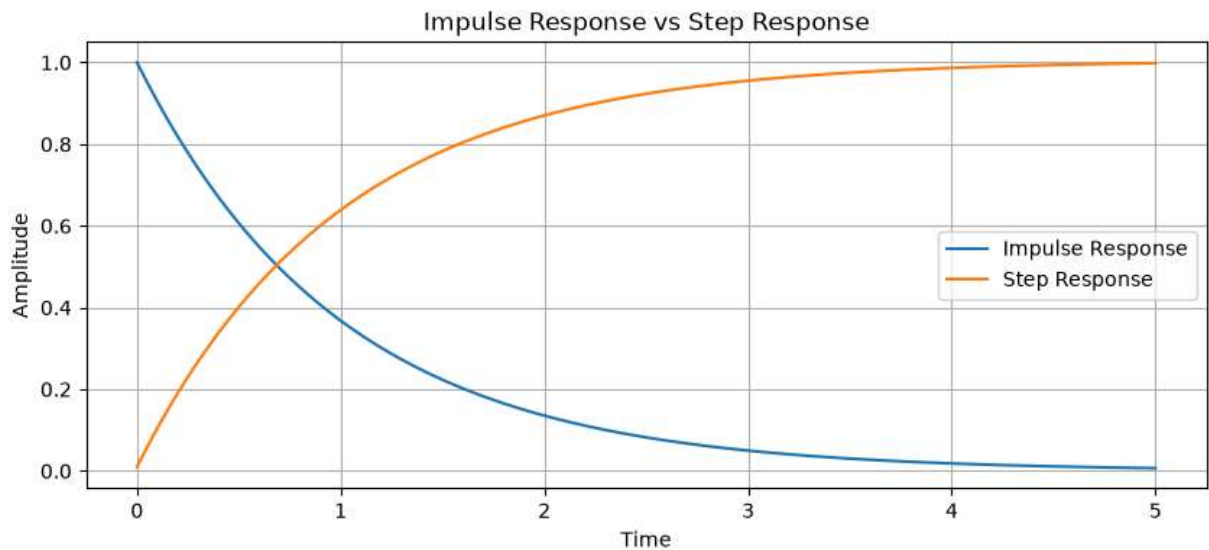
plt.xlabel("Time")
```

```
plt.ylabel("Amplitude")

plt.legend()

plt.grid(True)

plt.show()
```



Observation

- The step response is obtained by integrating the impulse response.
- The step response gradually reaches its steady-state value.
- The impulse response decays with time, whereas the step response accumulates over time.

Problem 12

Problem Statement

Verify that differentiating the step response approximately reproduces the impulse response.

```
In [20]: import numpy as np
import matplotlib.pyplot as plt

t=np.linspace(0,5,500)

dt=t[1]-t[0]

h=np.exp(-t)

step=np.cumsum(h)*dt

recovered=np.gradient(step,dt)
```

```
plt.figure(figsize=(10,4))

plt.plot(t,h,label="Original Impulse Response")

plt.plot(t,recovered,'--',label="Recovered from Step Response")

plt.title("Verification of Impulse-Step Relationship")

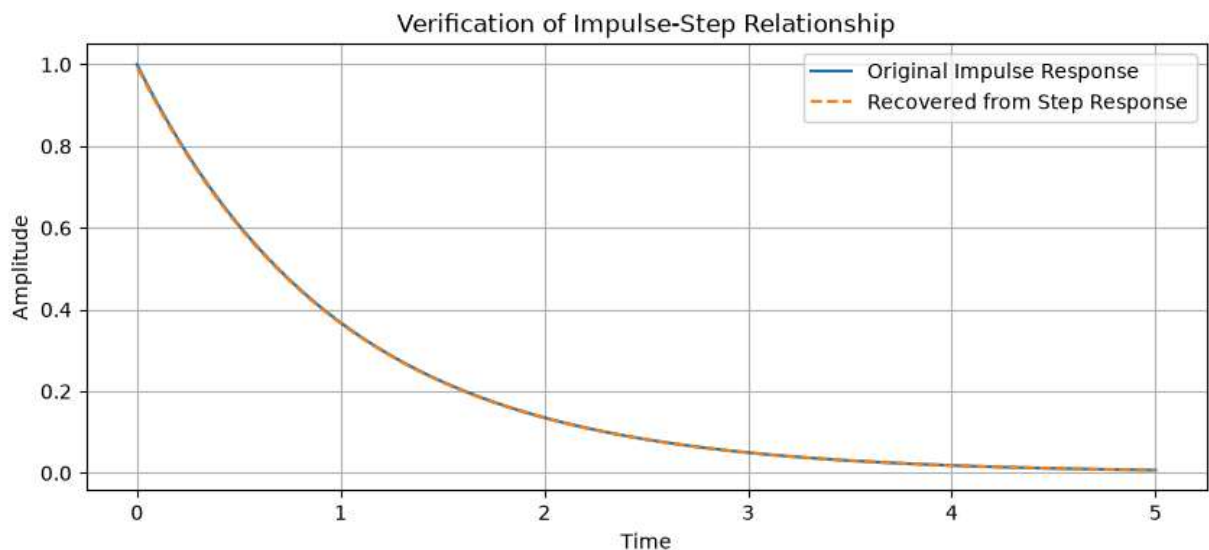
plt.xlabel("Time")

plt.ylabel("Amplitude")

plt.legend()

plt.grid(True)

plt.show()
```



Observation

- Differentiating the step response reproduces the impulse response with minor numerical approximation errors.
- This verifies the mathematical relationship between impulse response and step response.

Conclusion

The graphical and analytical methods of convolution produced identical results. The relationship between impulse response and step response was also verified. The impulse response completely characterizes an LTI system, while the step response can be obtained by integrating the impulse response.